

PixelPusher Bridge

Pont Art-Net / sACN vers PixelPusher, avec interface web complète

VERSION 1.6.0

macOS + WINDOWS

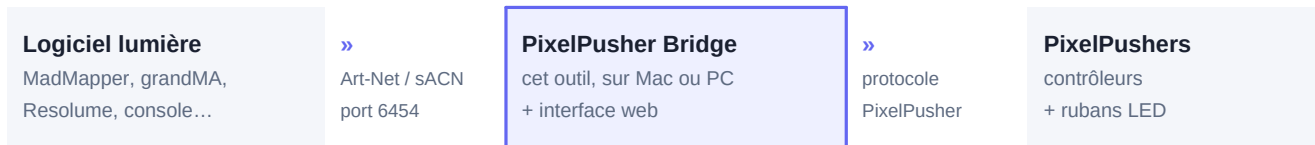
INSTALLATION EN 2 CLICS

[Présentation](#) · [Installation](#) · [Premiers pas](#) · [Dépannage](#)

PRÉSENTATION

Qu'est-ce que PixelPusher Bridge ?

PixelPusher Bridge est le maillon entre ton logiciel lumière (MadMapper, grandMA, Resolume, console, médiaserveur...) et tes contrôleurs PixelPusher qui pilotent les rubans LED. Il reçoit le signal Art-Net ou sACN sur le réseau et le transmet aux pushers, en continu et sans saccade. Il doit simplement rester lancé tant que les LED doivent répondre.



Les points forts

- **Fluidité d'abord** Le cœur réseau est le bridge open source éprouvé de Heroic Robotics, quasiment intact. L'interface, l'enregistreur et les outils tournent dans des threads séparés : zéro impact sur le flux LED.
- **Fiabilité vérifiée** La version 1.6.0 est issue d'un audit complet : 90 défauts identifiés et traités, dont six capables d'interrompre une représentation. **313 tests automatisés** et un test de bout en bout qui contrôle les couleurs réellement reçues par les LED tournent avant chaque publication.
- **Installation en 2 clics** Aucune configuration manuelle : sur Mac, l'app télécharge même Java toute seule si besoin. Sur PC, un double-clic suffit.
- **Une vraie app de bureau** Icône de barre système avec point vert (bridge en marche) : clic droit pour ouvrir l'interface, blackout, redémarrer ou arrêter. Fermer la fenêtre ne coupe pas le bridge.
- **Téléphone en un scan** Bouton « Afficher le QR code » sur le tableau de bord : scanne-le avec l'appareil photo et l'interface de contrôle s'ouvre — blackout, luminosité, tests dans la poche.
- **Pensé pour le spectacle** Blackout d'urgence verrouillé, limite de puissance électrique, watchdog de signal, presets par salle, lecture de séquences en boucle sans console.
- **Auto-dépannage** Diagnostic complet en un clic avec un conseil concret par problème, moniteur DMX en direct, guide intégré pour néophyte, rapport téléchargeable pour se faire aider.

FONCTIONNALITÉS

Ce que l'interface te permet de faire

Tableau de bord	Vue d'ensemble en temps réel : la check-list « Est-ce que tout marche ? », les débits Art-Net avec courbes, les univers actifs, la consommation électrique, chaque PixelPusher détecté (IP, lignes, FPS, univers utilisés), et le QR code téléphone.
Blackout verrouillé	Nouveau en 1.6.0. Le bouton Blackout ne se contente pas d'éteindre : il ignore les données entrantes jusqu'à une reprise explicite. Sans cela, la trame suivante de la console rallumait tout 25 ms plus tard — le bouton d'urgence était sans effet tant que la source tournait.
Limite de puissance	Nouveau en 1.6.0. Renseigne l'ampérage de ton alimentation : au-delà, toutes les LED sont atténuées proportionnellement plutôt que de laisser l'alim s'effondrer (chute de tension, couleurs qui virent, protection qui coupe). Jauge de consommation en direct.
Diagnostic en 1 clic	Passe tout en revue (réseau, pushers, données, configuration, système). Chaque problème vient avec sa solution — il repère les univers que ta source envoie mais qu'aucun pusher n'écoute, et détecte qu'un autre logiciel occupe déjà le port Art-Net.
Tests intégrés	10 scénarios sans logiciel externe : blanc 100 % (test d'alim), cycle RVB (ordre des couleurs), dégradé (sens des rubans), chenillard (pixels morts), 1 couleur par ligne (câblage)... et le test de lignes manuel, un bouton par sortie de chaque pusher.
Séquences	Enregistre ce que ta console envoie (tous les univers, timing exact) puis rejoue-le à l'identique, en boucle si besoin : l'installation tourne seule, sans console.
Presets	Photographie toute la configuration sous un nom (« Salle A », « Tournée »...) et recharge-la en un clic, à chaud.
Adressage DMX	Calculateur intégré : type de LED, LED par barre, barres par ligne, adresses de départ » la map complète barre par barre, fidèle au mapping réel du bridge. Export CSV.
Moniteur DMX	Les 512 canaux d'un univers en barres colorées, en direct : tu vois exactement ce que ta source envoie, canal par canal.
Sécurité spectacle	Watchdog : si la source coupe, blackout automatique après N secondes (réarmement auto, inhibé pendant une séquence ou un test). Blackout à l'arrêt.
Téléphone	Scanne le QR code du tableau de bord (même WiFi) : interface tactile simplifiée — blackout, luminosité, tests rapides, presets, séquences.
Logs & guide	Journal temps réel filtrable et téléchargeable, messages du cœur réseau traduits en français avec un conseil concret ; onglet Guide pas-à-pas pour démarrer de zéro.

INSTALLATION

Sur Mac (Apple Silicon et Intel)

- 1 Dézippe le fichier**
Double-clique **PixelPusher Bridge (macOS).zip** — l'app apparaît, accompagnée du script optionnel de signature.
- 2 Range l'app (optionnel)**
Glisse-la dans le dossier **Applications**.
- 3 Premier lancement**
Clic droit sur l'app » Ouvrir » Ouvrir. macOS demande confirmation une seule fois car l'app n'est pas signée Apple. Ensuite : double-clic normal.
- 4 Java (automatique)**
Si Java n'est pas présent, l'app propose de le télécharger toute seule (~45 Mo, une seule fois, connexion internet requise).
- 5 Autorisation « Réseau local »**
Sur **macOS 15 (Sequoia) et plus récent**, le système la demande au premier lancement : **accepte**. C'est elle qui permet de découvrir les PixelPushers.
- 6 C'est lancé**
L'interface s'ouvre dans une fenêtre dédiée, une icône apparaît dans la barre de menus (point vert = en marche). Fermer la fenêtre ne coupe pas le bridge.

Aucun PixelPusher détecté sur un Mac récent ? Vérifie l'autorisation **Réglages Système » Confidentialité et sécurité » Réseau local** et active *PixelPusher Bridge*. Un refus est **totalemtent silencieux** : aucune erreur, simplement plus aucun pusher trouvé alors que tout paraît normal.

« **L'app est endommagée** » (fréquent après un transfert par internet) : double-clique le script *Signer l'app (optionnel).command* fourni dans le zip, ou colle dans le Terminal : `xattr -cr "/Applications/PixelPusher Bridge.app"`

INSTALLATION

Sur PC Windows

- 1 Ouvre le dossier**
PixelPusher Bridge (Windows) — copie-le où tu veux (Bureau, Documents...).
- 2 Lance l'app**
Double-clique **PixelPusher Bridge.bat**. Relancer l'app alors qu'elle tourne déjà rouvre simplement l'interface : jamais de doublon.
- 3 Pare-feu (1^{re} fois)**
Autorise Java sur les réseaux privés — indispensable pour détecter les PixelPushers et recevoir l'Art-Net.
- 4 Pour arrêter**
Clic droit sur l'icône de la barre système » **Arrêter**, ou le bouton Arrêter dans l'interface. En secours : *Arreter PixelPusher Bridge.bat*.

Prérequis : Java 11 ou plus récent. Le lanceur le vérifie réellement et ouvre adoptium.net si besoin (installation gratuite, 2 minutes). En cas d'échec, il laisse une trace dans `%USERPROFILE%\pixelpusherbridge\launcher.log`.

Mise à jour (Mac comme PC) : remplace simplement le fichier *PixelPusherBridge.jar*. Ta configuration, tes presets et tes séquences sont conservés.

PREMIERS PAS

De zéro jusqu'aux LED allumées

1

Branche le PixelPusher

Rubans sur les sorties du pusher, pusher sur le même réseau que l'ordinateur (câble Ethernet conseillé — le WiFi peut saccader). Sur sa carte SD, *pixel.rc* doit avoir **artnet_universe=1** et **artnet_channel=1** (ou plus) : à 0, rien ne s'allume.

2

Vérifie la détection

Lance le bridge : le pusher apparaît sur le Tableau de bord en moins de 10 secondes, « en ligne » et « mappé ». Sinon : bouton **Lancer le diagnostic**.

3

Teste sans logiciel

Onglet Tests : « Blanc 100 % », puis « Cycle RVB » — quand l'écran dit ROUGE, les LED doivent être rouges. Sinon : Configuration » Ordre des couleurs (effet immédiat).

4

Configure ta source

Protocole Art-Net, destination = l'IP de l'ordinateur qui fait tourner le bridge (ou broadcast), port 6454. **Décalage classique : univers 0 côté logiciel = univers 1 ici**. L'onglet Adressage DMX calcule la map exacte.

5

Protège ton alimentation

Configuration » Puissance électrique : renseigne l'ampérage de ton alim. Le bridge atténuera plutôt que de la laisser s'effondrer en plein spectacle.

6

En spectacle

Active le watchdog (5–30 s conseillé), sauvegarde un preset, et scanne le QR code pour garder blackout et luminosité sous la main sur ton téléphone.

DÉPANNAGE

Les symptômes les plus fréquents

Symptôme	Solution
Pusher non détecté	Alimentation ? Même réseau / VLAN ? Pare-feu : autoriser Java (UDP entrant 7331). Sur macOS 15+ : autorisation Réseau local .
Détecté mais « non mappé »	<code>artnet_universe</code> / <code>artnet_channel</code> à 0 dans <code>pixel.rc</code> » mettre 1 et redémarrer le pusher.
Données reçues, rien ne s'allume	Décalage d'univers 0/1 côté source, ou mauvais univers » le diagnostic le détecte.
Mauvaises couleurs	Test « Cycle RVB », puis Configuration » Ordre des couleurs.
LED anormalement sombres	Le limiteur de puissance est peut-être actif : la jauge du tableau de bord l'indique. Vérifie aussi la luminosité globale.
Saccades	Câble plutôt que WiFi ; sinon +1–2 ms de délai additionnel, ou active l'auto-throttle (le diagnostic le conseille quand un pusher le réclame).
L'interface est injoignable	Un pare-feu ou un antivirus bloque les connexions locales dont Java a besoin. Le bridge bascule tout seul sur son serveur de secours et le signale dans les logs ; autorise Java pour revenir au fonctionnement normal.
Le port Art-Net ne reçoit rien	Un autre logiciel de la machine occupe déjà le port 6454 (MadMapper, Resolume, un autre node). Le diagnostic le dit explicitement.
Image figée après fermeture	Normal : les LED gardent la dernière trame. Active « Blackout à l'arrêt » ou clique Blackout.

Infos pratiques

Interface web	<code>http://localhost:7350</code> sur la machine du bridge — port modifiable dans Configuration.
Téléphone	QR code du tableau de bord, ou l'adresse courte affichée à côté (même réseau WiFi).
Tes données	Dossier <code>.pixelpusherbridge</code> (configuration, presets, séquences, journaux) — conservé lors des mises à jour.
En cas de problème	Tableau de bord » Rapport : fichier de diagnostic complet à envoyer pour se faire aider.
Sources & mises à jour	<code>github.com/pymenvert/pixelpusherbridge</code>
Licence	Logiciel libre (MIT) — © 2026 Pierre Yves Mansour — Collectif WSK. Cœur réseau crédité à Heroic Robotics / robot-head.